MODULE 2 · CORE WORKLOADS

Rollouts & Rollbacks

Changing a Deployment safely — and backing out when it goes wrong

WHY IT MATTERS

New image, zero downtime — that's the whole point of a rollout

Editing a Deployment's Pod template doesn't touch existing Pods directly. It triggers a **rolling update**: Kubernetes stands up Pods on the new template and retires old ones gradually, so the app keeps serving traffic throughout.

Lab 2.3 built the Deployment. This lab is about **changing it safely** — and knowing exactly how to back out if the new version is bad.

BY THE END YOU CAN

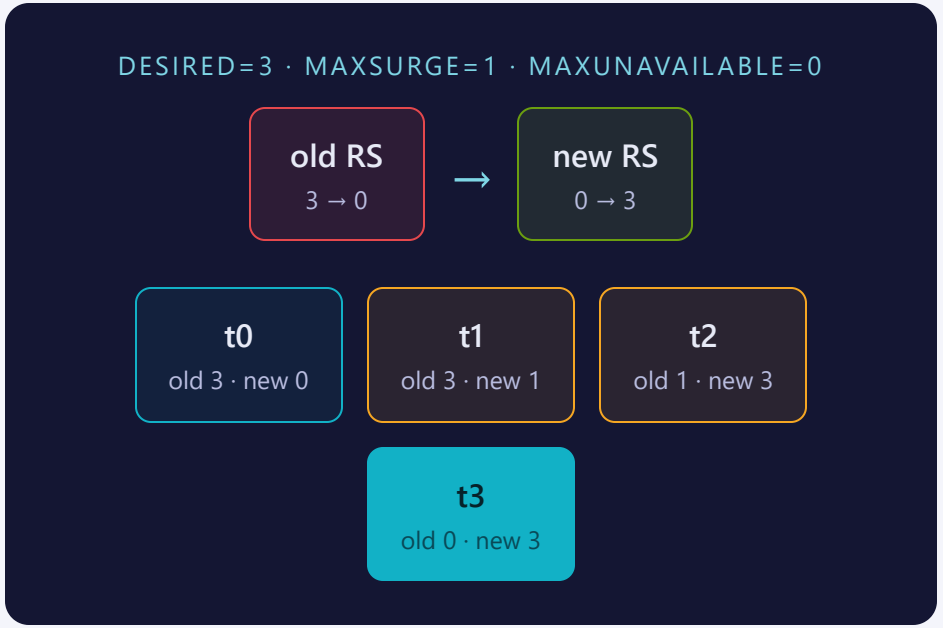
- Trigger a rolling update
- Watch it converge with `rollout status`
- Read revision history
- Roll back with `rollout undo`

CONCEPT

A Deployment never edits Pods — it swaps ReplicaSets

Changing the Pod template creates a brand-new **ReplicaSet**. The Deployment then shifts replica counts in lockstep: scale the new ReplicaSet up, scale the old one down, until only the new one is left.

The old ReplicaSet isn't deleted — it's kept at **0 replicas** so a rollback has something to scale straight back up.



ROLLING UPDATE MECHANICS

Two knobs set the pace of the swap

SETTING	CONTROLS	DEFAULT
`maxSurge`	Extra Pods allowed above `replicas` while updating	25%
`maxUnavailable`	How many Pods may be unready during the update	25%

MENTAL MODEL

`maxSurge` trades extra capacity for speed; `maxUnavailable` trades availability for speed. `maxUnavailable: 0` guarantees full capacity throughout — but only if there's room to schedule the surge Pod.

MANIFEST ANATOMY

The full Deployment — strategy block included

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rolling-nginx
  namespace: training
  labels:
    app: rolling-nginx
spec:
  replicas: 3
  strategy:
    type: RollingUpdate          # default; alt: Recreate
    rollingUpdate:
      maxSurge: 1              # extra Pods allowed above 3
      maxUnavailable: 1        # Pods allowed unready
  selector:
    matchLabels:
      app: rolling-nginx
  template:
    metadata:
      labels:
        app: rolling-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.24    # the field step 4 changes
          ports:
            - containerPort: 80
```

This is `labs/2.4/deployment.yaml` in full. **strategy.type** is `RollingUpdate` (default) or `Recreate` (kill all, then start new — downtime); **maxSurge**/**maxUnavailable** here override the 25%/25% default; the **image** line is what `kubectl set image` patches.

DO IT — TWO WAYS

Patch the image, or edit and reapply

IMPERATIVE — FAST, ONE FIELD	DECLARATIVE — VERSIONABLE
`kubectl set image deployment/rolling-nginx nginx=nginx:1.25`	Edit `image:` in `deployment.yaml`, then `kubectl apply -f deployment.yaml`
Patches the live object right now	Reviewable diff; git stays the source of truth

THE BRIDGE TRICK

Generate YAML from an imperative patch, then edit & keep it:

```
$ kubectl set image deployment/rolling-nginx nginx=nginx:1.25 \
    --dry-run=client -o yaml > deployment.yaml
```

SEE IT

`rollout status` turns a black box into a progress bar

The command **blocks** until the rollout finishes, fails, or times out — it doesn't just poll and print once.

That makes it safe to drop straight into a pipeline: a non-zero exit means the rollout didn't converge, and something needs attention before you move on.

```
$ kubectl rollout status deployment/rolling-nginx -n training
Waiting for deployment "rolling-nginx" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "rolling-nginx" rollout to finish: 2 out of 3 new replicas have been updated...
deployment "rolling-nginx" successfully rolled out
```

REVISION HISTORY

History remembers templates — and it dedupes

Every rollout writes a new entry, backed by the ReplicaSet it created. `kubectl rollout history` lists revisions; add `--revision=N` to see that template in full.

Rolling back re-numbers. `rollout undo` doesn't restore the old revision number — it moves that ReplicaSet's template to the top as a brand-new, higher revision. The number you rolled back *from* disappears from history.

WATCH

Never hardcode a revision number in a script. Read `rollout history` first, then pick a revision that's actually still there.

ALSO TRUE

`revisionHistoryLimit` caps how many old ReplicaSets are kept around (default 10) — beyond that, they're garbage-collected.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **`Recreate` strategy.** Kills every old Pod before starting new ones — real downtime, not a rolling update.
- **`maxUnavailable: 0` with no headroom.** If the cluster can't schedule the surge Pod, the rollout stalls `Pending` forever.
- **Assuming revision numbers persist.** They don't — `rollout undo` dedupes and renumbers history.
- **No change-cause annotation.** Without `kubernetes.io/change-cause`, history shows `<none>` — no clue what each revision was for.

YOUR ROLLOUT TOOLKIT

Commands to keep at hand

```
$ kubectl set image deployment/NAME c=IMG:TAG      # trigger a rolling update
$ kubectl rollout status deployment/NAME           # block until it converges
$ kubectl rollout history deployment/NAME          # list revisions
$ kubectl rollout history deployment/NAME --revision=N   # inspect one
$ kubectl rollout undo deployment/NAME             # back to the previous revision
$ kubectl rollout undo deployment/NAME --to-revision=N   # back to a specific one
$ kubectl annotate deployment/NAME kubernetes.io/change-cause="..."  # label a revision
```

Tip: confirm with `rollout status` before you undo — don't roll back a healthy update that's just still in progress.

RECAP

Rollouts & rollbacks in one breath

- A rolling update swaps ReplicaSets — it never edits Pods in place
- `maxSurge` / `maxUnavailable` set the pace (default 25% / 25%)
- `rollout status` watches progress; `rollout history` lists revisions
- `rollout undo` restores an old template — and renumbers it

HANDS-ON

Now run `Lab 2.4`

`labs/2.4/`

Next: **2.5 — Probes**